

Install Required Libraries

```
pip install pandas numpy matplotlib scikit-learn
```

Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
```

Task 1: Load FER2013 Dataset

Python Code

```
import pandas as pd

# Load FER2013 dataset
df = pd.read_csv("fer2013.csv")

# Display first five rows
print(df.head())

# Total samples
print("Total Samples:", len(df))

# Unique emotion classes
print("Unique Emotion Classes:")
print(df["emotion"].unique())
```

Example Output

Total Samples: 35887

Unique Emotion Classes:
[0 1 2 3 4 5 6]

Emotion Labels

Label Emotion

0 Angry

Label Emotion

- 1 Disgust
 - 2 Fear
 - 3 Happy
 - 4 Sad
 - 5 Surprise
 - 6 Neutral
-

Task 2: Count Samples per Emotion Class

```
emotion_counts = df["emotion"].value_counts().sort_index()

print("Samples per Emotion:\n")
print(emotion_counts)
```

Example Output

Emotion Samples

- 0 4953
- 1 547
- 2 5121
- 3 8989
- 4 6077
- 5 4002
- 6 6198

Visualize Class Distribution

```
emotion_counts.plot(
    kind="bar",
    figsize=(8,5))
```

```
)

plt.title("FER2013 Emotion Distribution")
plt.xlabel("Emotion")
plt.ylabel("Number of Samples")
plt.show()
```

Observation

The **Disgust (label 1)** class has significantly fewer samples than the others, indicating class imbalance.

Task 3: Convert Pixel String to Image

Each image is stored as a space-separated string of **2304 values (48 × 48)**.

```
def pixel_to_image(pixel_string):
```

```
    pixels = np.array(
        pixel_string.split(),
        dtype=np.float32
    )
```

```
    image = pixels.reshape(48,48)
```

```
    return image
```

Test Function

```
sample_pixels = df.iloc[0]["pixels"]
```

```
image = pixel_to_image(sample_pixels)
```

```
print("Image Shape:", image.shape)
```

Output

```
Image Shape:
(48, 48)
```

Display Image

```
plt.imshow(image, cmap="gray")
```

```
plt.title("FER2013 Image")
```

```
plt.axis("off")
```

```
plt.show()
```

Task 4: Normalize and Split Dataset

Convert All Images

```
images = np.array(  
    df["pixels"].apply(  
        lambda x: np.fromstring(x, sep=" ")  
    ).tolist()  
)  
  
labels = df["emotion"].values
```

Normalize

```
images = images / 255.0
```

Reshape

```
images = images.reshape(-1,48,48,1)  
  
print(images.shape)
```

Train (80%)

```
X_train, X_temp, y_train, y_temp = train_test_split(  
    images,  
    labels,  
    test_size=0.20,  
    random_state=42,
```

```
stratify=labels  
)
```

Validation (10%) + Test (10%)

```
X_val, X_test, y_val, y_test = train_test_split(  
  
    X_temp,  
  
    y_temp,  
  
    test_size=0.50,  
  
    random_state=42,  
  
    stratify=y_temp  
)
```

Check Shapes

```
print("Training :", X_train.shape)  
  
print("Validation :", X_val.shape)  
  
print("Testing :", X_test.shape)
```

Example Output

Training:
(28709,48,48,1)

Validation:
(3589,48,48,1)

Testing:
(3589,48,48,1)

Task 5: Handle Class Imbalance

Since **Disgust** has very few samples, one common approach is **oversampling** the minority class so that it appears more frequently during training.

Example Using Random Oversampling

```
from sklearn.utils import resample
import pandas as pd

# Convert training labels to DataFrame
train_df = pd.DataFrame({
    "label": y_train
})

# Find minority class
minority = train_df[train_df.label == 1]

# Oversample minority class
minority_upsampled = resample(
    minority,
    replace=True,
    n_samples=4000,
    random_state=42
)

print("Original Disgust Samples:", len(minority))
print("After Oversampling:", len(minority_upsampled))
```

Alternative Approach (Recommended for Deep Learning)

Instead of duplicating images, you can use **data augmentation** to create varied versions of minority-class images.

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator

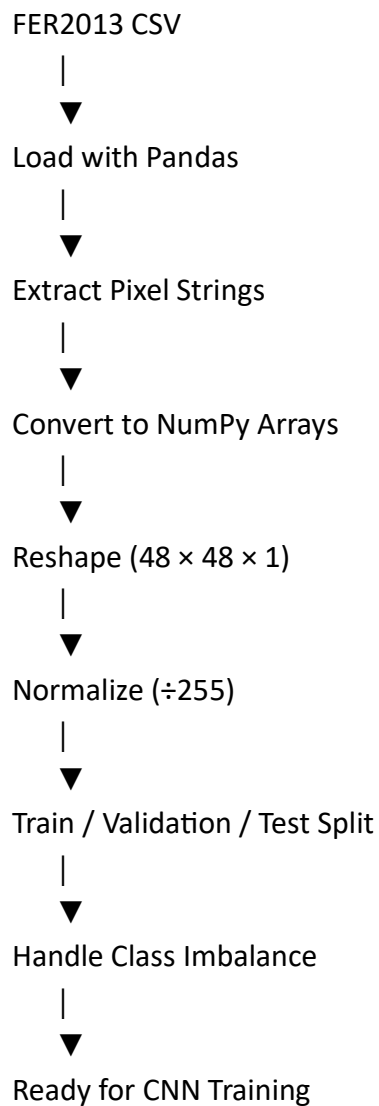
datagen = ImageDataGenerator(
    rotation_range=20,
    zoom_range=0.2,
    width_shift_range=0.1,
    height_shift_range=0.1,
    horizontal_flip=True
)
```

```
# Example:
# augmented_images = datagen.flow(
#   X_train[y_train == 1],
#   batch_size=32
# )
```

Why This Helps

Data augmentation creates new variations of existing **Disgust** images (rotated, shifted, flipped, etc.), which increases the diversity of the minority class without simply duplicating the same samples. This can help the model generalize better and reduce bias toward the majority classes.

FER2013 Data Pipeline



Final Summary

Task Implementation

Key Learning

- | | | |
|----------|---|--|
| 1 | Loaded the FER2013 CSV with pandas and listed the unique emotion labels. | The dataset contains 35,887 grayscale face images labeled with 7 emotion classes . |
| 2 | Counted samples per emotion using <code>value_counts()</code> and visualized the distribution. | Reveals class imbalance, especially the underrepresented Disgust class. |
| 3 | Converted a pixel string into a NumPy array and reshaped it to 48 × 48 . | FER2013 stores each image as a space-separated string of 2304 pixel values . |
| 4 | Normalized pixel values to [0, 1] and split the data into 80% training, 10% validation, and 10% testing . | Normalization and proper data splitting are essential preprocessing steps before training a CNN. |
| 5 | Demonstrated oversampling and data augmentation for the minority Disgust class. | Balancing classes helps improve model performance and reduces bias toward majority classes. |